



Asset Documentation

Asset Development & Documentation Writing by: **Saad “Bunssar” Abounasr**

Section I: Asset Overview & Face Animator

1. Overview:

Matthew is a 3D model of a stylized character, it comes rigged, textured, and animated.

The expressive nature of the motions and countenances, combined with the heavily stylized shader make it a perfect fit for cartoony, whimsical games. And with its user-friendly design, you can easily customize it to fit your game: Change main colors and accent colors at runtime, retexture the clothes, modify individual body parts, change facial features, you name it!

This asset offers numerous interesting systems that you can play, so make sure you refer to this documentation when messing around lest you break the code or get confused. Enjoy!



Matthew idling

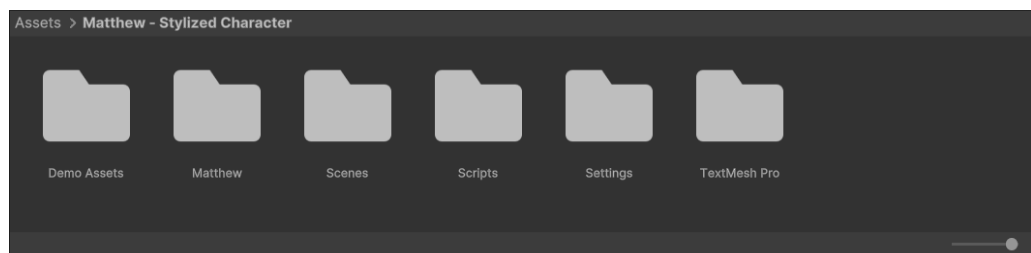


Matthew in a T-pose

2. Quick Start:

Once you import the asset, you'll be met with a folder named "Matthew - Stylized Character". This folder contains a set of subfolders, each containing a different category of files.

- **Demo Assets:** This folder contains bonus assets used to build the demo scenes. Things like showcase scripts, demo structures, and materials.
- **Matthew:** The cream of the crop; It contains the most essential parts of the asset, as the name suggests, it holds Matthew's model, textures, animations, face data, materials, and more.
- **Scenes:** Here you'll find the demo scenes. They showcase the various features of the asset.
- **Scripts:** Another essential folder. Within are the scripts for the facial animation system.
- **Settings:** The URP asset, renderer, global settings, and the post processing profile, are all in here.
- **TextMesh Pro:** To handle text rendering. It's an official Unity package.



The main folder

Inside the "Matthew" folder, you'll find a subfolder named "Prefab". This subfolder contains a ready-to-use prefab of Matthew, with Face Animator already set up and an Animator with a demo controller assigned that cycles through all the animations.

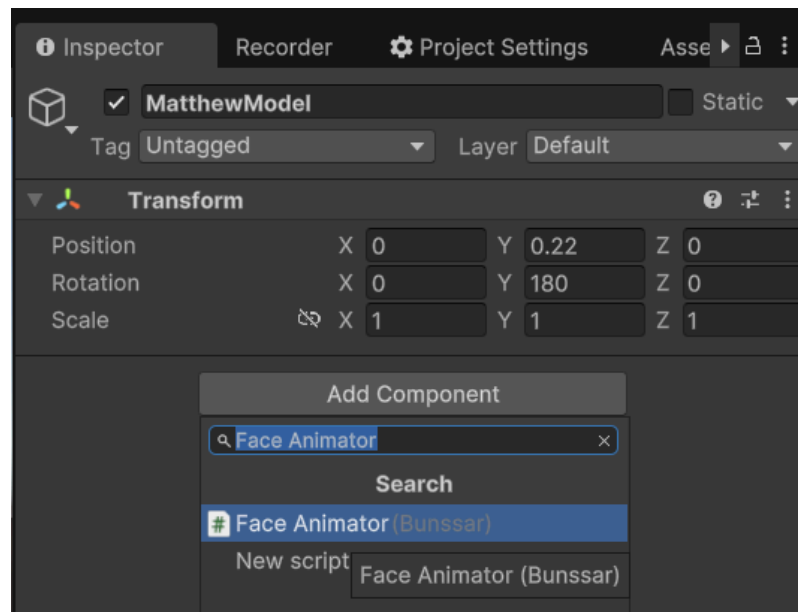
You can find more detailed instructions below if you need them.

3. Setting Up Face Animator:

If you need to set up Face Animator manually, follow these steps:

Step 1: Add The Component

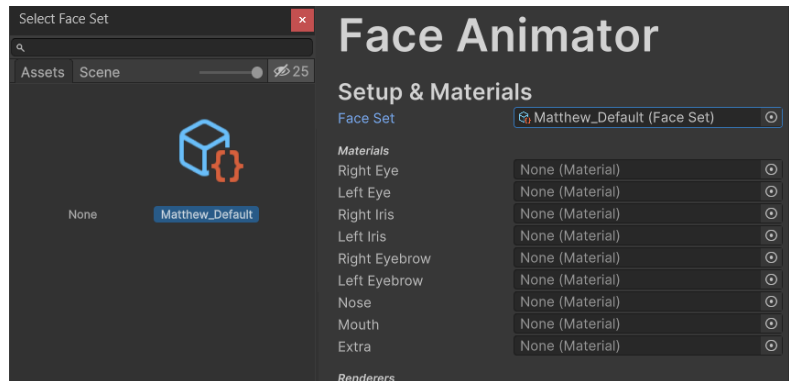
Simply add the “Face Animator” component to the “MatthewModel” GameObject.



Component added

Step 2: Assign a Face Set

Assign a Face Set to the “Face Set” slot. You can either use the included default Face Set, or create your own by right clicking the in the “Project” panel, and navigating to “Create>Bunssar>Facial Animation>Face Set”. You can find instructions on how to configure Face Sets in **Step 4**.

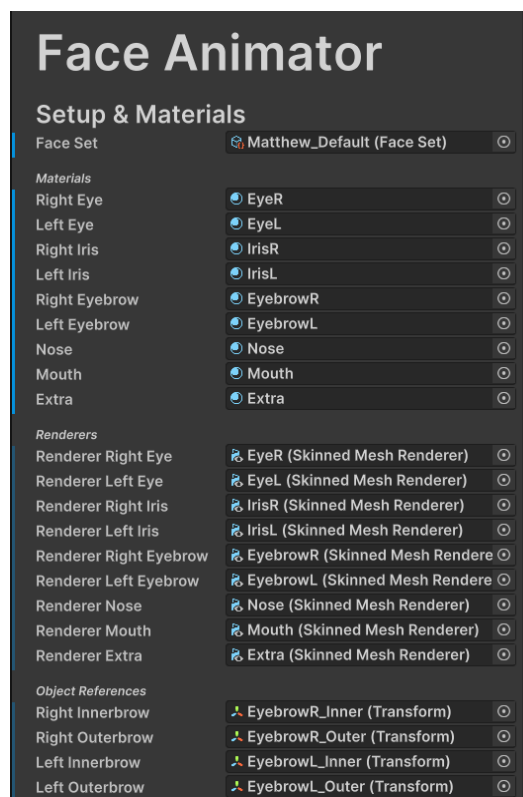


Face set assigned

Step 2: Assign The Proper Materials/Renderers/Transforms:

You'll find that the Face Animator component has plenty of empty slots for materials, renderers, and transforms. Assigning them is pretty straightforward:

- **Materials:** Assign the corresponding materials from the "Matthew>Materials" folder.
- **Renderers:** Assign the corresponding skinned mesh renderers from the scene (Children of "MatthewModel").
- **Object References:** Assign the bones corresponding to each empty slot. They are named as: EyebrowX_Y (X: R or L - Y: Inner or Outer).



All slots assigned

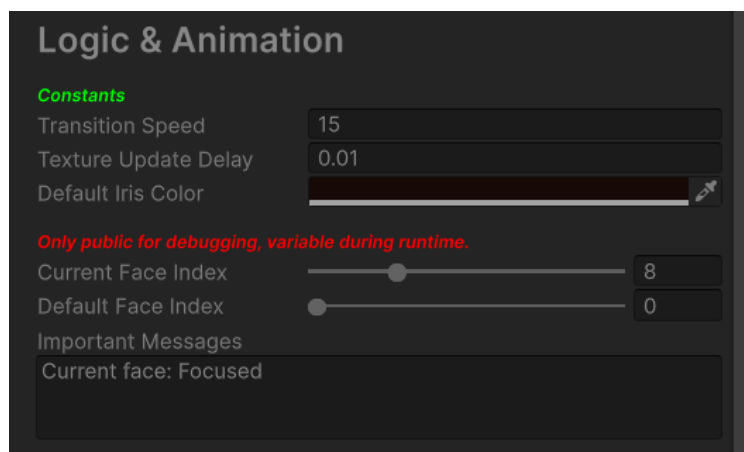
Step 3: Logic & Animation

There is an extra category at the very bottom named “Logic & Animation”. Here you’ll find the variables that you can use to control Face Animator.

- **Transition Speed:** Speed of the transition between faces (Recommended value: 15).
- **Texture Update Delay:** This allows you to set a delay between the animation’s start and the textures updating, resulting in a smoother transition (Recommended value: 0.01)
- **Default Iris Color:** The fallback color of the character’s irises when “Override Iris Color” in the current face element is false. (Recommended value: #220E09)

The next values are only public to make debugging/testing easier. They allow you to quickly test faces without having to code or set up a test animation.

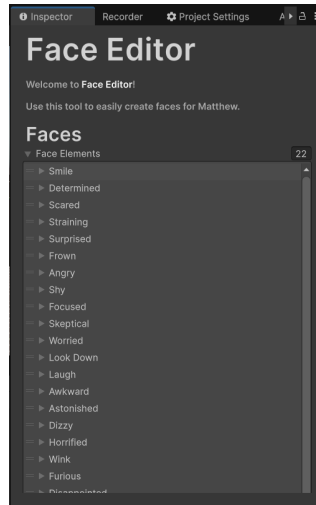
- **Current Face Index:** The index of the face element you want to visualize.
- **Default Face Index:** Face Animator allows you to set a default face as a universal fallback when calling the `ReturnToDefaultFace()` function. This is the index of that face element.
- **Important Messages:** This text area displays useful information, such as the current face, or if the requested face is out of bounds.



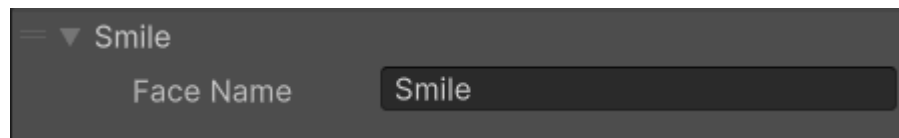
Example

Step 4: Creating Faces:

Creating faces for Matthew is very easy using the Face Editor. To edit/create faces, select a Face Set from the “Project” panel, and you’ll be met with a convenient GUI. Here’s some more info about what each button/value does:

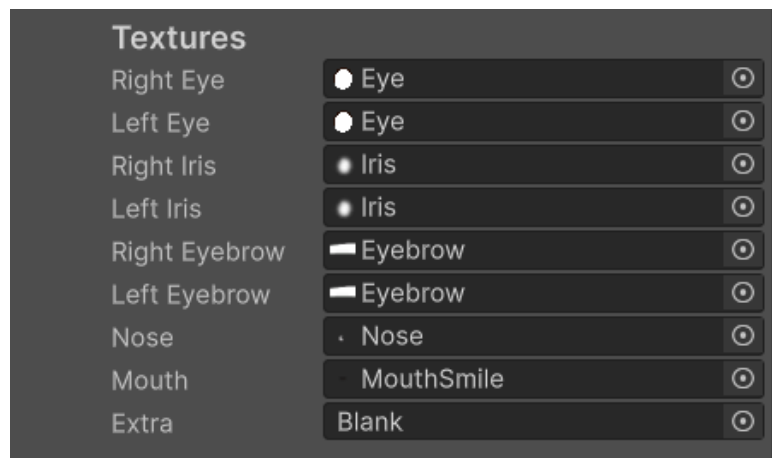


Face Editor GUI



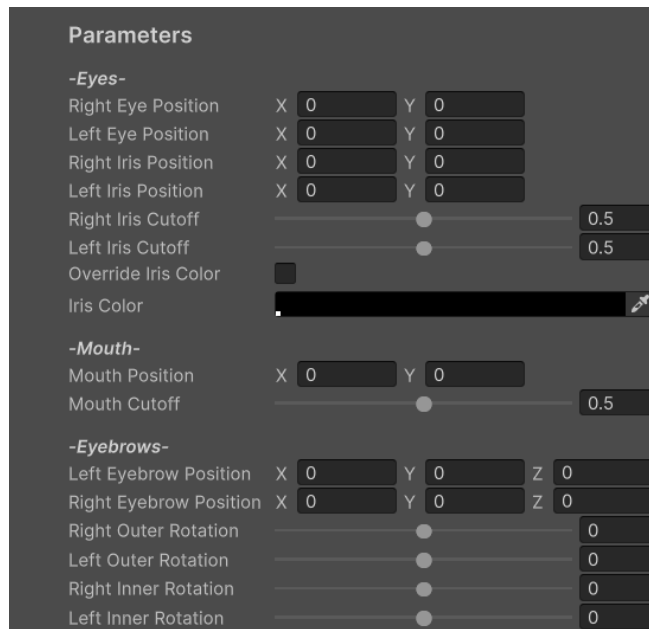
Face Name

- **Face Name:** As the name suggests, this is the name of the facial expression.



Texture section

- **Textures:** Assign the texture you want to use for each facial element (textures are updated when the CurrentFaceIndex integer in Face Animator is changed)



Parameter section

1. -Eyes-:

- Right/Left Eye Position controls the position of the eye.
(X: higher value moves outward | Y: higher value moves lower)
- Right/Left Iris Position controls the position of the iris.
(X: higher value moves outward | Y: higher value moves lower)
- Right/Left Iris Cutoff controls the alpha clipping threshold of the irises.
(higher value shrinks the irises)
- Override Iris Color enables you to use a different iris color from the default color set in Face Animator.
(true uses the Iris Color from the Face Element, false uses the default color from Face Animator)
- Iris Color is the override color for the irises, used when Override Iris Color is true.

2. -Mouth-:

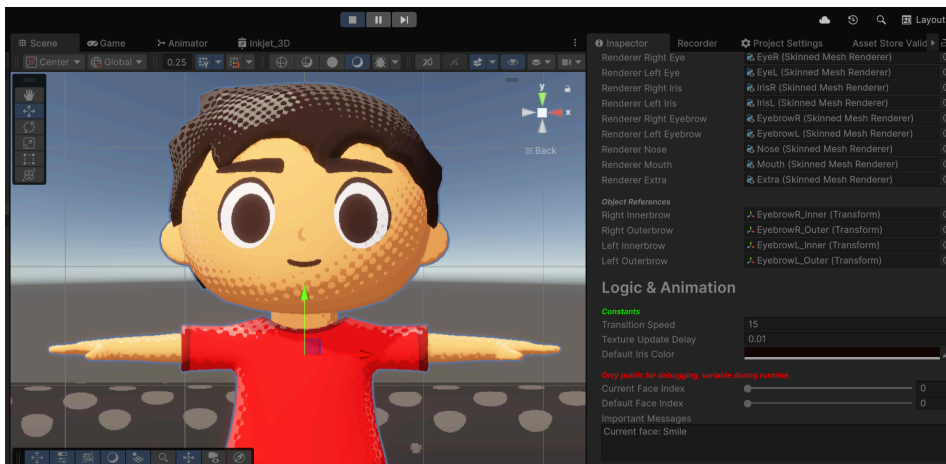
- Mouth Position changes the position of the mouth.
(X: higher value moves to the local right | Y: higher value moves lower)
- Mouth Cutoff controls the alpha clipping threshold of the mouth.
(higher value shrinks the mouth)

3. -Eyebrows-:

- Left/Right Eyebrow Position controls the position of the eyebrows.
(X: higher value moves to the local left | Y: lower value moves to the local forward | Z: higher value moves lower)

- Right/Left Outer Rotation controls the base rotation of the eyebrows.
(higher value rotates upwards)
- Right/Left Inner Rotation controls the inner rotation of the eyebrows.
(higher value rotates upwards)

After creating your face, enter Play Mode and select its index in Face Animator. You should see your face in action.



Face applied

Note: The minimum value for the cutoffs is 0.05, this is because a cutoff of 0 disables alpha clipping, nullifying the purpose of the cutoff.

4. Triggering Transitions

Face Animator is designed to work with Unity's animation system. You can use Animation Events or call the C# methods with a UI button, a script, etc...

Here are the available methods for Face Animator:

- SetFace(int FaceIndex)
- ReturnToDefaultFace()
- SetDefaultFaceTo(int FaceIndex)

SetFace sets the current face to the FaceIndex parameter, ReturnToDefaultFace falls back to the default face, and SetDefaultFaceTo allows you to set a default face.

If you're debugging or testing, the Current Face Index and Default Face Index sliders in the inspector are also an option to trigger transitions without writing code or setting up an animation.

Section II: The Inkjet3D Shader

1.Quick Start:

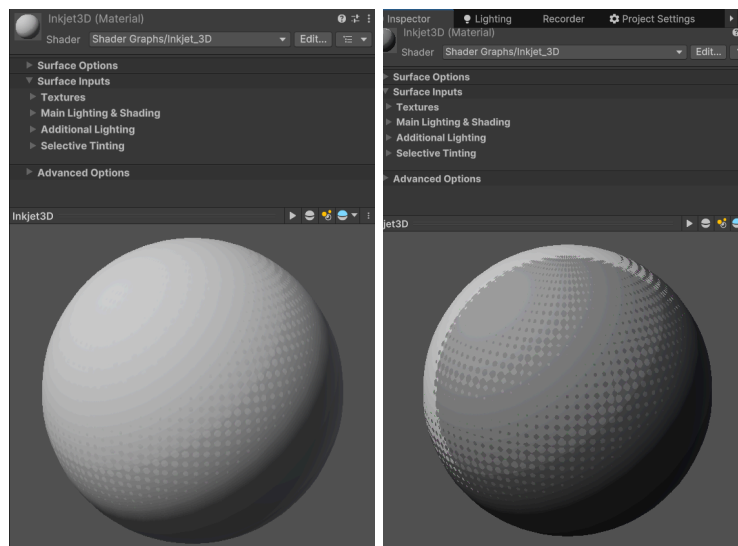
Stylization is easier than ever now with the Inkjet3D shader, a versatile, easily modifiable solution for realtime toon 3D shading.

To start using Inkjet3D, simply navigate to “Matthew>Shader”, right click the “Inkjet3D” shadergraph, and choose “Create>Material”. This will create a new material with the shader, it comes already configured, so you can just drag and drop the material onto any mesh renderer and it will work right away.

Note: To maintain a certain degree of detail, and to also enable support for things like smoothness, metallic, and reflections, the Inkjet3D shader overlays the stylized shading over the existing one of the “Lit” shader. For best results, I recommend setting the ambient color in the lighting settings to a brighter color.

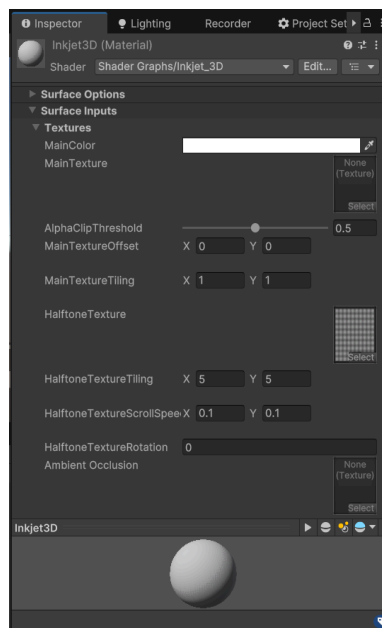
2.Configuring The Shader:

Inkjet3D is made to be customizable, there are plenty of variables to tweak, let’s dive into what each one does.



1. Textures:

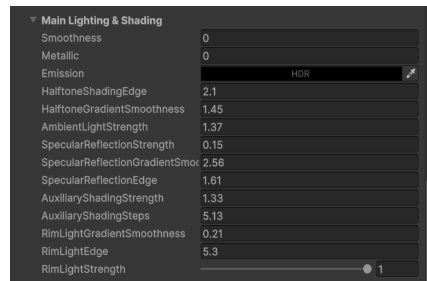
- MainColor: This is the main tint that will be applied to the material, or the MainTexture if it is assigned.
- MainTexture: The main texture of the material.
- AlphaClipThreshold: The clipping threshold for the MainTexture if it includes transparency.
- MainTextureOffset: Offsets the main texture by the X & Y coordinates.
- MainTextureTiling: Sets the tiling of the texture to a X*Y grid.
- HalftoneTexture: The pattern used for shading. This **MUST** be a monochrome, blurred pattern (reasoning available later on in the documentation).
- HalftoneTextureTiling: Sets the tiling of the texture to a X*Y grid.
- HalftoneTextureScrollSpeed: The scrolling rate of the shading texture. Adds a dynamic aspect to the shader. Set both components to 0 to disable.
- HalftoneTextureRotation: Rotation of the shading texture in degrees.
- AmbientOcclusion: Texture to use for AO.



Texture section

2. Main Lighting & Shading:

- Smoothness: Reflectivity of the material.
- Metallic: Metallic-ness of the material.
- Emission: Emission color, useful to create a glowing material.
- HalftoneShadingEdge: Controls the edge of the halftone shading.
- HalftoneGradientSmoothness: Smoothens the halftone shading. (higher value results in a longer gradient)
- AmbientLightStrength: Strength of ambient lighting. (0 means pitch black shadows)
- SpecularReflectionStrength: Strength of the specular highlight.
- SpecularReflectionGradientSmoothness: Smoothens the specular reflection. (higher value results in a longer gradient)
- SpecularReflectionEdge: Controls the edge of the specular highlight.
- AuxiliaryShadingStrength: Strength of the auxiliary shading. (the posterized bands of shading)
- AuxiliaryShadingSteps: Number of bands the auxiliary shading has. (higher value results in more bands in the gradient)
- RimLightGradientSmoothness: Smoothens the rim light (higher value results in a smoother rim light)
- RimLightEdge: Controls the edge of the rim light.
- RimLightStrength: Strength of the rim light.



Main lighting & shading section

3. Additional Lighting:

- AdditionalLightGradientSmoothness: Smoothens the additional light's shading.
- AdditionalLightAttenuationStrength: Determines the attenuation of the additional light. (lower value means the light fades out faster with distance)
- AdditionalLightEdge: Controls the edge of the additional light.
- AdditionalLightStrength: Strength of the additional light.



Additional lighting section

4. Selective Tinting:

- Enabled: Whether Selective Tinting is enabled or not.
- SecondaryColorMask: Mask used for Selective Tinting. White areas in the mask get colored with SecondaryColor, black areas aren't affected at all. Shades of gray dictate influence. (closer to white means more influence)
- SecondaryColor: The color used to tint the areas marked with SecondaryColorMask.

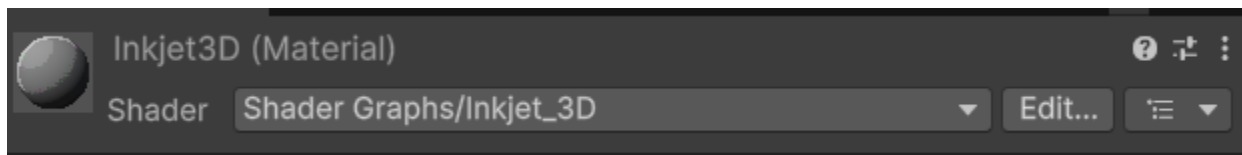


Selective tinting section

3.Presets:

Inkjet3D comes with some presets in the “Matthew>Shader>Presets” folder. These are simple examples of different Inkjet3D configurations meant to serve as a base for you to expand upon.

To use a preset, simply press the button next to “?”, the one with the two sliders, and choose a preset.



Presets included with Inkjet3D

4.Behind The Scenes (Custom Shading Patterns):

You can make custom shading patterns for Inkjet3D if the included dots, lines, and ripples are not enough. A custom shading pattern should follow these criteria:

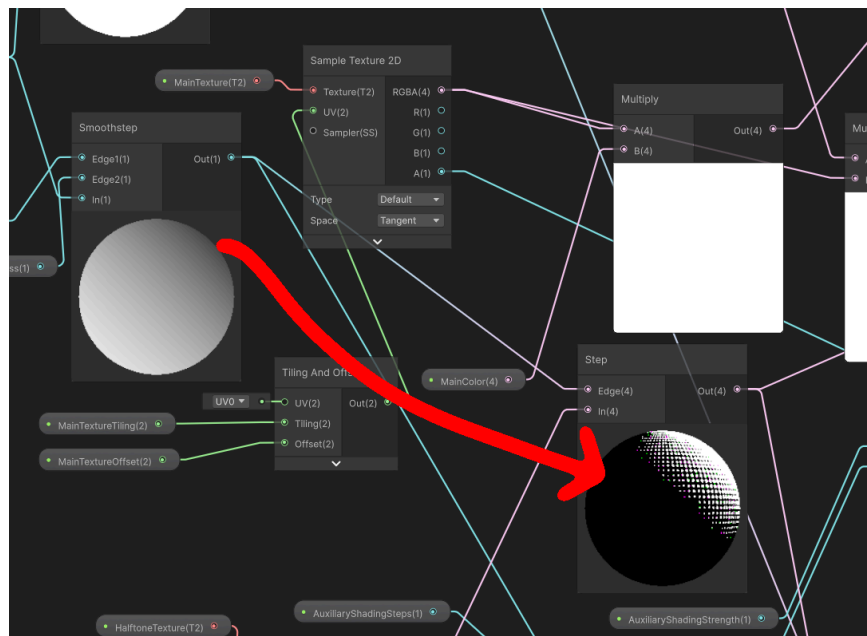
- It should be monochrome. (black, white, and shades of gray)
- It should be blurred.
- Preferably seamless and tiles well.



Examples (included)

• Why a blurred monochrome texture?

Inkjet3D sets a threshold for the value of the shading texture, essentially forcing every pixel to be either black or white, rounding the gray values up or down. Whether a pixel is rounded to black or white depends on the dot product calculation of the lighting. If a pixel is on a face that points towards the light source, then the rounding will be aggressive to 0, making the pattern invisible in the brightest areas, and the opposite occurs for when a face points away from the light source; the rounding is aggressive to 1, making the pattern fully solid in the darkest areas. The areas in between will essentially have either a higher or a lower pattern density depending on the normal of the face in question.



Instance of the process in shadergraph

Section III: FAQ (Important Notes)

This section includes frequently asked questions and some very important notes on using the different systems. I highly recommend reading it as it clears confusion in many aspects of the package.

- **Q: Does Face Animator allow using the same materials for several instances of Matthew?**

A: Yes, you can have as many instances of Matthew as your computer can handle, and they can all have different faces. This is because Face Animator creates a new instance of the assigned materials, and sets those cloned materials to the Matthew instance it's connected to, essentially creating a set of proprietary materials exclusive to each character. This is what allows for different facial expressions simultaneously on different characters without the need of manually creating and assigning materials.

- **Q: Does Inkjet3D support additional lights?**

A: Yes, Inkjet3D supports 1 additional light alongside the main light.

- **Q: Can I use any animation with Matthew?**

A: To some extent. Since Matthew has a humanoid rig, he is technically compatible with any humanoid animation, but since he's of cartoony proportions, animations made for realistic humanoids might look off. For best results, use the included animations, or get animations designed for cartoony characters.

- **Q: How many faces does Face Animator support?**

A: Face Animator supports as many faces as Unity can handle, so you don't have to worry about running out of space. Make as many faces as you like!

- **Q: Is this asset compatible with all render pipelines?**

A: No, this asset was made in URP and is only compatible with URP.

- **Q: Do I have to credit you when using this asset?**

A: No, you can use it without giving me credit. It is, however, greatly appreciated.

- **Q: How can I contact you?**

A: I'm active on Instagram (@bun.ssar) and on Discord (bunssar). You can contact me if you have questions about the asset, or if you simply want to chat with me.

And Finally...

I hope this documentation was of help to you. I'd love to see what you create with Matthew, so let's start a hashtag. Post your creations on Instagram with the hashtag #MadeWithMatthew and tag me (@bun.ssar). Have fun!